

MANUAL DE DESARROLLO



Proyecto Kaz Motors

[Guía para desarrolladores](#)

Antonio Jesús Castillo Henares

TFG - 2025/26

IES Gran Capitán - Córdoba

1. MANUAL DE DESARROLLO.....	1
Proyecto Kaz Motors.....	1
2. Introducción técnica.....	4
3. Arquitectura general.....	4
1. Visión global.....	4
1. Flujo de una petición típica.....	5
4. Stack tecnológico.....	5
1. Backend.....	5
3. App Android.....	6
5. Backend con Spring Boot.....	6
1. Estructura de paquetes.....	6
2. Modelo de datos.....	6
3. Cardinalidades principales.....	7
4. Entidad de ejemplo: MotoNueva.....	8
5. Capa de servicio.....	9
6. Capa de controlador.....	9
7. Seguridad y autenticación.....	10
Componentes implicados.....	10
Flujo de autenticación.....	10
8. Configuración de SecurityConfig.....	10
9. Generación del token en AuthController.....	11
10. Envío del token desde Android.....	12
11. Almacenamiento de contraseñas.....	13
12. 4.5.7 Roles del sistema.....	13
2. 4.6 Gestión de archivos estáticos.....	13
6. Aplicación Android.....	15
1. Patrón MVVM aplicado.....	15
2. Comunicación con la API.....	15
3. Gestión de imágenes con Glide.....	16
4. Reproductor de vídeo con ExoPlayer.....	17
5. Vista 360°.....	18
6. Búsqueda tolerante (Levenshtein).....	18
7. Decisiones técnicas relevantes.....	20
1. Por qué JWT con BCrypt.....	20
2. ¿Por qué Docker Compose?.....	20
3. ¿Por qué ngrok?.....	20
4. ¿Por qué ExoPlayer en lugar de YouTube?.....	20
1. ¿Por qué OpenMultipleDocuments en lugar de GetMultipleContents?.....	20
5. ¿Por qué desactivar Auto Backup?.....	21
8. Guía del programador.....	21
9. Configuración del entorno de desarrollo.....	21
1. Software necesario.....	21
1. Clonar el proyecto.....	21
2. Levantar el backend en modo desarrollo.....	21
3. Abrir la app Android.....	21
10. Cómo añadir un nuevo módulo completo.....	22
1. Backend: repositorio.....	22
2. Backend: DTO.....	22
3. Backend: servicio.....	22
4. Backend: controlador.....	23
1. Backend: configurar seguridad.....	23
5. Android: data class y endpoints.....	23

6. Android: repositorio, ViewModel y Fragments.....	23
11. Cómo añadir una nueva pantalla en Android.....	24
1. Cambios sin pérdida de datos.....	25
2. Cambios destructivos.....	25
1. Borrar todos los datos y empezar de cero.....	25
12. Buenas prácticas y debugging.....	26
1. Logs útiles.....	26
2. 13.2 Errores frecuentes.....	26
1. Cleartext HTTP not permitted.....	26
2. Error 401/403 al acceder a un endpoint.....	26
3. Error 500 al subir archivos.....	26
4. OutOfMemoryError al cargar imágenes.....	26
13. Despliegue.....	27
1. Generar APK firmado de release.....	27

1. Introducción técnica

Este documento describe la arquitectura, las decisiones de diseño y la organización del código del proyecto, así como las pautas a seguir para retomar su desarrollo o ampliarlo con nuevas funcionalidades. Está dirigido a desarrolladores con conocimientos previos de programación orientada a objetos, Kotlin, Android y Spring Boot.

El proyecto consiste en un sistema cliente-servidor compuesto por dos elementos:

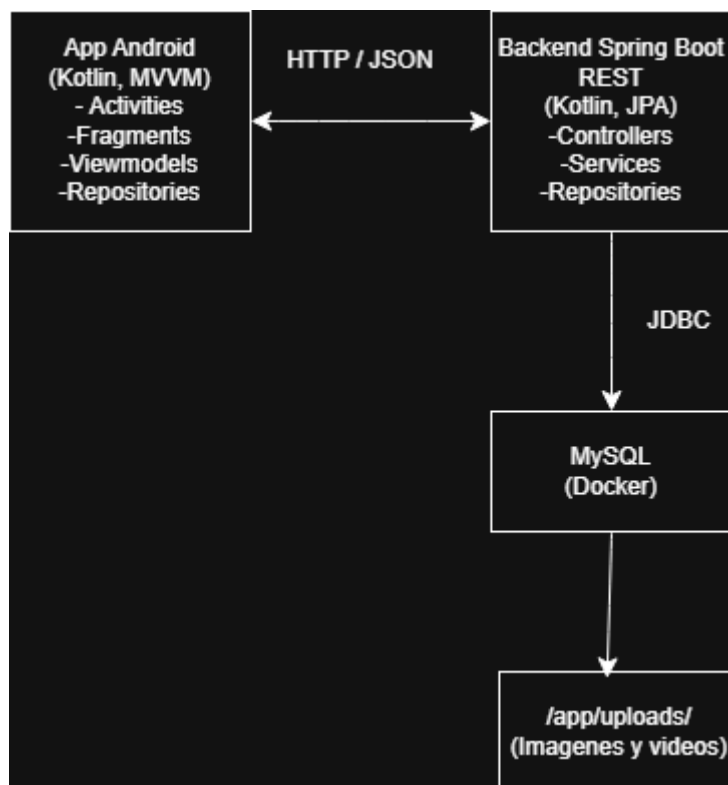
- Un backend REST construido con Spring Boot 3 y Kotlin que gestiona toda la lógica de negocio y la persistencia de datos.
- Una aplicación móvil Android nativa, también escrita en Kotlin, que sigue el patrón arquitectónico MVVM.

Ambos componentes se comunican mediante peticiones HTTP/HTTPS sobre una API REST con autenticación basada en JSON Web Tokens (JWT) y contraseñas cifradas con BCrypt. La base de datos es MySQL 8 y todo el backend se despliega mediante Docker Compose.

2. Arquitectura general

1. Visión global

El sistema sigue una arquitectura en tres capas físicas:



1. Flujo de una petición típica

Cuando el usuario realiza una acción que requiere datos del servidor (por ejemplo, listar motos nuevas), el flujo es el siguiente:

1. El Fragment captura el evento y llama al método correspondiente del ViewModel.
2. El ViewModel llama al Repository (en background usando coroutines).
3. El Repository llama al ApiService de Retrofit.
4. Retrofit envía una petición HTTP autenticada al backend.
5. Spring Security valida las credenciales y los permisos del rol.
6. El Controller recibe la petición y delega en el Service.
7. El Service aplica la lógica de negocio y consulta el Repository JPA.
8. Hibernate ejecuta la consulta SQL contra MySQL.
9. La respuesta sube por las capas hasta convertirse en JSON.
10. Retrofit deserializa el JSON a objetos Kotlin.
11. El Repository devuelve los datos al ViewModel.
12. El ViewModel actualiza un LiveData.
13. El Fragment, que observa ese LiveData, refresca la UI.

3. Stack tecnológico

1. Backend

Tecnología	Versión	Función
Kotlin	1.9	Lenguaje principal
Spring Boot	3.x	Framework principal
Spring Web	3.x	Endpoints REST
Spring Data JPA	3.x	Persistencia
Spring Security	6.x	Autenticación y autorización
Hibernate	6.x	ORM (implementación de JPA)
MySQL	8.x	Base de datos relacional
Docker / Compose	v2	Contenedorización
Gradle Kotlin DSL	8.x	Build tool

2. App Android

Tecnología	Versión	Función
Kotlin	1.9	Lenguaje principal
Android SDK	36 (compile)	Min SDK 26 (Android 8)
AndroidX	varios	Lifecycle, Fragment, Navigation
Material Components	1.12	Diseño Material 3
Retrofit	2.11	Cliente HTTP
OkHttp	4.12	Capa de red
Glide	4.15	Carga y caché de imágenes
ExoPlayer (Media3)	1.4	Reproducción de vídeo
View Binding	nativo	Acceso tipado a vistas

4. Backend con Spring Boot

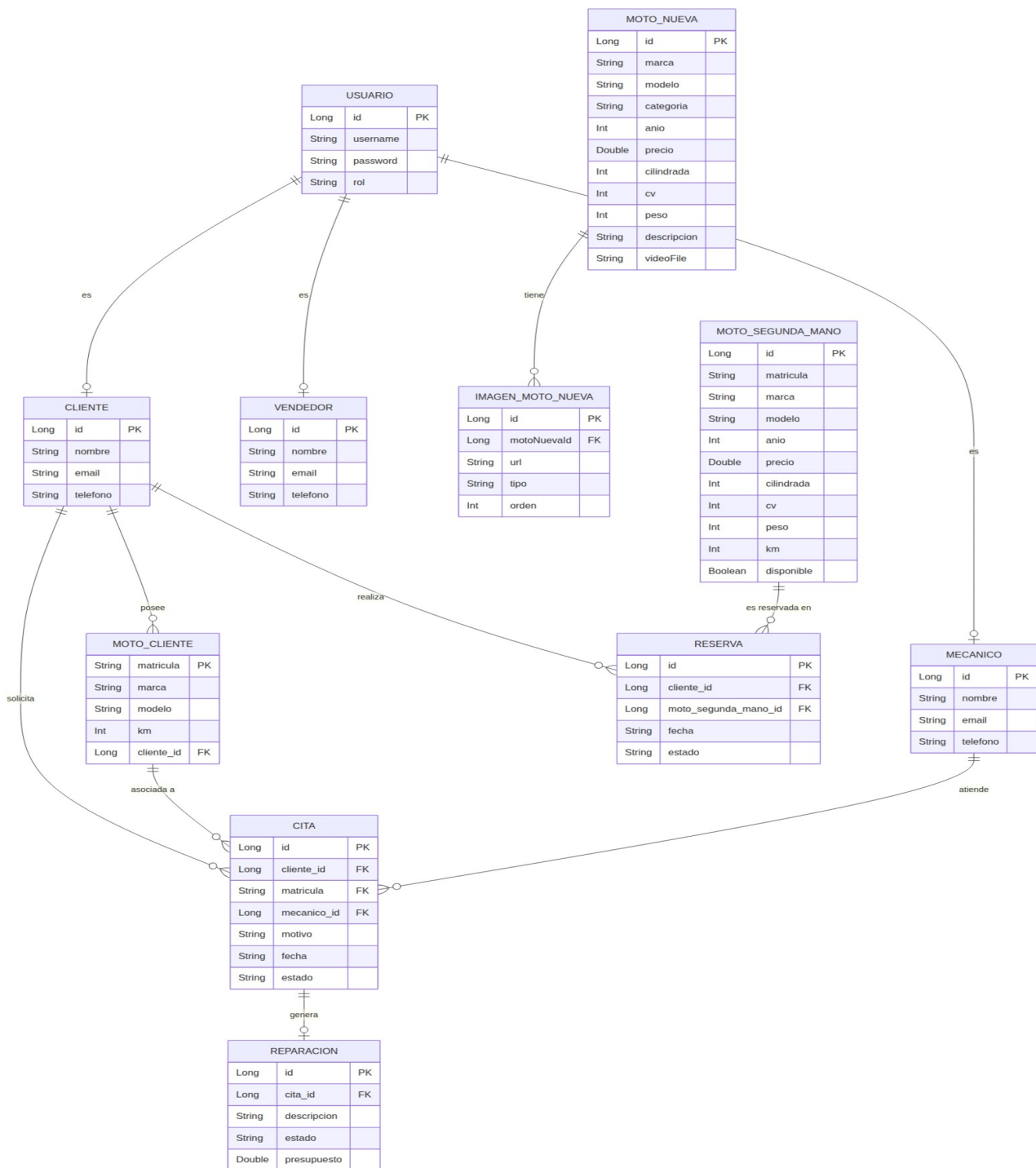
1. Estructura de paquetes

Sigue una organización por capas:

▼ kotlin\com\tfg\concesionario	
▼ config	
SecurityConfig.kt	Configuración de Spring Security
WebConfig.kt	Resource handles(uploads/imagenes/videos)
> controller	Endpoints REST
> dto	Objetos de transferencia
> model	Entidades JPA
> repository	Acceso a la BD (JPA)
> security	
> service	Logica de negocio
ConcesionarioApiApplication.kt	
> resources	

2. Modelo de datos

El esquema relacional del proyecto se compone de doce entidades principales que cubren los tres dominios funcionales del sistema: usuarios y empleados, catálogo de motocicletas y operativa del taller. La siguiente figura muestra el diagrama entidad-relación completo con todas las cardinalidades:



3. Cardinalidades principales

- Usuario (1) ↔ (0..1) Cliente / Mecánico / Vendedor: cada usuario está vinculado a un único registro del rol que le corresponde.
- Cliente (1) ↔ (N) MotoCliente: un cliente puede tener varias motos en su perfil. Cada moto pertenece a un único cliente.
- Cliente (1) ↔ (N) Cita: un cliente puede solicitar múltiples citas a lo largo del tiempo.
- Cliente (1) ↔ (N) Reserva: un cliente puede reservar varias motos de segunda mano (en distintos momentos).
- MotoCliente (1) ↔ (N) Cita: una moto puede tener varias citas a lo largo de su vida útil.

- Mecánico (1) ↔ (N) Cita: un mecánico atiende varias citas. Cada cita la atiende un único mecánico.
- Cita (1) ↔ (1) Reparación: cada cita confirmada genera una única reparación con su descripción y estado.
- MotoNueva (1) ↔ (N) ImagenMotoNueva: cada modelo del catálogo tiene varias imágenes asociadas, distinguidas por tipo (GALERIA o R360).
- MotoSegundaMano (1) ↔ (N) Reserva: una moto puede haber sido reservada varias veces si las anteriores fueron rechazadas o canceladas.

4. Entidad de ejemplo: MotoNueva

Cada entidad utiliza anotaciones JPA estándar. Por ejemplo, MotoNueva.kt:

```
@Entity
@Table(name = "moto_nueva")
data class MotoNueva(
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    val id: Long = 0,

    @Column(nullable = false)
    val marca: String = "",

    @Column(nullable = false)
    val modelo: String = "",

    @Column(nullable = false)
    val categoria: String = "",

    @Column(nullable = false)
    val anio: Int = 0,

    @Column(nullable = false)
    val precio: Double = 0.0,

    @Column(nullable = false)
    val cilindrada: Int = 0,

    @Column(nullable = false)
    val cv: Int = 0,

    @Column(nullable = false)
    val peso: Int = 0,

    @Column(columnDefinition = "TEXT")
    val descripcion: String = "",

    @Column(length = 500)
    val videoFile: String? = null
)
```

5. Capa de servicio

Los servicios encapsulan la lógica de negocio. Reciben los repositorios por inyección de dependencia y devuelven entidades o DTOs. Ejemplo simplificado de MotoNuevaService:

```
@Service
class MotoNuevaService(
    private val repo: MotoNuevaRepository,
    private val imgRepo: ImagenMotoNuevaRepository
) {
    fun getAll(): List<MotoNueva> = repo.findAll()

    fun get(id: Long): MotoNueva = ...

    fun getByMarca(marca: String): List<MotoNueva> = repo.findByMarca(marca)

    fun getByMarcaCategoria(marca: String, categoria: String): List<MotoNueva> = ...

    fun save(request: MotoNuevaRequest): MotoNueva = ...

    fun update(id: Long, request: MotoNuevaRequest): MotoNueva { ...
    }

    fun delete(id: Long) {
        imgRepo.findByMotoNuevaIdOrderByOrdenAsc(id).forEach { imgRepo.deleteById(it.id) }
        repo.deleteById(id)
    }
}
```

6. Capa de controlador

Los controllers exponen los endpoints REST y delegan toda la lógica en los servicios:

```
@RestController
@RequestMapping("/motosNuevas")
class MotoNuevaController(private val service: MotoNuevaService) {

    @GetMapping
    fun getAll(): List<MotoNueva> = service.getAll()

    @GetMapping("/{id}")
    fun get(@PathVariable id: Long): MotoNueva = service.get(id)

    @GetMapping("/marca/{marca}")
    fun getByMarca(@PathVariable marca: String): List<MotoNueva> = service.getByMarca(marca)

    @GetMapping("/marca/{marca}/categoria/{categoria}")
    fun getByMarcaCategoria( ...
    ): List<MotoNueva> = service.getByMarcaCategoria(marca, categoria)

    @PostMapping
    fun create(@RequestBody request: MotoNuevaRequest): MotoNueva = service.save(request)

    @PutMapping("/{id}")
    fun update(@PathVariable id: Long, @RequestBody request: MotoNuevaRequest): MotoNueva = ...

    @DeleteMapping("/{id}")
    fun delete(@PathVariable id: Long) = service.delete(id)
}
```

7. Seguridad y autenticación

La autenticación del sistema se basa en JSON Web Tokens (JWT) con cifrado BCrypt para las contraseñas almacenadas en base de datos. La configuración es completamente stateless: el servidor no mantiene sesión por usuario, lo que permite escalar horizontalmente y simplifica el despliegue.

Componentes implicados

- BCryptPasswordEncoder: cifra las contraseñas con sal aleatoria antes de guardarlas en la BD.
- JwtService: genera tokens firmados al hacer login y valida los recibidos en cada petición.
- JwtFilter: filtro que se ejecuta antes del filtro de autenticación de Spring Security para extraer y validar el token de cada petición.
- SecurityConfig: configura las reglas de autorización por rol y conecta el JwtFilter a la cadena de filtros.

Flujo de autenticación

El proceso completo desde que el usuario hace login hasta que accede a un recurso protegido es el siguiente:

14. El cliente envía username y password al endpoint /auth/login.
15. El backend valida la contraseña comparándola con el hash BCrypt almacenado en la BD.
16. Si es correcta, JwtService genera un token firmado con HMAC-SHA256 que contiene el username y el rol como claims.
17. El backend devuelve el token al cliente, junto con el rol y los identificadores específicos (clienteId, mecanicoId, vendedorId).
18. El cliente guarda el token en SessionManager (SharedPreferences).
19. En cada petición posterior, el AuthInterceptor de OkHttp añade la cabecera: Authorization: Bearer <token>.
20. En el backend, JwtFilter intercepta la petición, extrae el token de la cabecera, valida su firma y caducidad, y carga la información del usuario en el contexto de seguridad de Spring.
21. Spring Security comprueba que el rol del usuario coincide con los permitidos para la ruta solicitada.
22. Si todo es correcto, el controller procesa la petición. Si no, devuelve 401 (no autenticado) o 403 (sin permisos).

8. Configuración de SecurityConfig

Extracto de la configuración real de seguridad. Cada ruta se protege con los roles permitidos:

```

@Bean
fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {

    http
        .csrf { it.disable() }

        .sessionManagement {
            it.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        }

        .authorizeHttpRequests {
            it
                .requestMatchers("/auth/**").permitAll()
                .requestMatchers("/uploads/**").permitAll()
                .requestMatchers("/usuarios/**").hasRole("ADMIN")
                .requestMatchers("/clientes/**").hasAnyRole("ADMIN", "CLIENTE", "VENDEDOR")
                .requestMatchers("/motos/**").permitAll()
                .requestMatchers("/motoCliente/**").hasAnyRole("ADMIN", "CLIENTE", "MECANICO")
                .requestMatchers("/imagenes/promociones/**").permitAll()
                .requestMatchers("/imagenes/promomecanico/**").permitAll()
                .requestMatchers("/imagenes/motosegundamano/**").permitAll()
                .requestMatchers("/imagenes/upload").hasAnyRole("ADMIN", "VENDEDOR")
                .requestMatchers("/imagenes/**").permitAll()
                .requestMatchers("/motosSegundaMano/**").permitAll()
                .requestMatchers("/mecanicos/**").hasAnyRole("ADMIN", "MECANICO")
                .requestMatchers("/vendedores/**").hasAnyRole("ADMIN", "VENDEDOR")
                .requestMatchers("/reservas/**").hasAnyRole("ADMIN", "CLIENTE", "VENDEDOR")
                .requestMatchers("/ventas/**").hasAnyRole("ADMIN", "VENDEDOR")
                .requestMatchers("/citas/**").hasAnyRole("ADMIN", "CLIENTE", "MECANICO", "VENDEDOR")
                .requestMatchers("/reparaciones/**").hasAnyRole("ADMIN", "MECANICO", "CLIENTE")
                .requestMatchers("/motosNuevas/**").permitAll()
                .requestMatchers("/imagenesMotoNueva/moto/**").permitAll()
                .requestMatchers("/imagenesMotoNueva/upload").hasAnyRole("ADMIN", "VENDEDOR")
                .requestMatchers("/imagenesMotoNueva/**").hasAnyRole("ADMIN", "VENDEDOR")
                .requestMatchers("/videosMotoNueva/**").hasAnyRole("ADMIN", "VENDEDOR")
                .anyRequest().authenticated()
        }
    }
}

```

La política STATELESS desactiva el uso de sesiones de servidor: cada petición debe portar su propio token JWT. `addFilterBefore` inserta el `JwtFilter` antes del filtro de autenticación estándar de Spring, lo que permite identificar al usuario a partir del token.

9. Generación del token en AuthController

El controller de autenticación valida las credenciales y genera el token al hacer login:

```

@PostMapping("/login")
fun login(@RequestBody request: LoginRequest): Map<String, Any?> {
    val usuario = usuarioService.getByUsername(request.username)
        ?: throw RuntimeException("Usuario no encontrado")

    if (!passwordEncoder.matches(request.password, usuario.password)) {
        throw RuntimeException("Contraseña incorrecta")
    }

    val token = jwtService.generateToken(
        username = usuario.username,
        rol = usuario.rol.name
    )

    val mecanicoId = if (usuario.rol == Rol.MECANICO) {
        mecanicoRepo.findAll().find { it.nombre == usuario.username }?.id
    } else null

    val vendedorId = if (usuario.rol == Rol.VENDEDOR) {
        vendedorRepo.findAll().find { it.nombre == usuario.username }?.id
    } else null

    return mapOf(
        "token" to token,
        "id" to usuario.id,
        "username" to usuario.username,
        "rol" to usuario.rol.name,
        "clienteId" to usuario.cliente?.id,
        "mecanicoId" to mecanicoId,
        "vendedorId" to vendedorId
    )
}

```

10. Envío del token desde Android

El cliente Android adjunta el token automáticamente en cada petición mediante un Interceptor de OkHttp:

```

class AuthInterceptor(private val session: SessionManager) : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val token = session.getToken()
        val request = if (token != null) {
            chain.request().newBuilder()
                .addHeader("Authorization", "Bearer $token")
                .build()
        } else {
            chain.request()
        }
        return chain.proceed(request)
    }
}

```

11. Almacenamiento de contraseñas

Las contraseñas se almacenan cifradas con BCrypt antes de persistirlas. BCrypt es un algoritmo de hash adaptativo diseñado específicamente para contraseñas, que añade automáticamente una sal aleatoria a cada hash y permite ajustar su coste computacional. Esto hace inviable el descifrado por fuerza bruta incluso si la base de datos quedara comprometida.

12. Roles del sistema

El sistema define cinco roles con permisos diferenciados:

Rol	Permisos
ADMIN	Acceso completo a todos los endpoints. Puede crear empleados.
VENDEDOR	Gestiona catálogos, sube imágenes/vídeos, atiende reservas.
MECANICO	Ve y actualiza citas y reparaciones asignadas.
CLIENTE	Gestiona su perfil, motos, citas y reservas.
INVITADO	Acceso al catálogo en modo solo lectura.

4.6 Gestión de archivos estáticos

Las imágenes y los vídeos no se almacenan en BD sino en el filesystem del contenedor, en la ruta /app/uploads/. Esta carpeta está montada como volumen Docker, por lo que persiste entre reinicios y se mapea al directorio uploads/ del proyecto en el host.

WebConfig expone el contenido como URL accesible:

```
@Configuration
class WebConfig : WebMvcConfigurer {

    override fun addResourceHandlers(registry: ResourceHandlerRegistry) {
        registry.addResourceHandler("/imagenes/**")
            .addResourceLocations("file:imagenes/")

        registry.addResourceHandler("/uploads/**")
            .addResourceLocations("file:/app/uploads/")
    }
}
```

Cuando el frontend necesita una imagen, construye la URL así:

<https://vendor-pasture-scalding.ngrok-free.dev/uploads/imagenes/motonueva/1234.jpg>

5. Aplicación Android

1. Patrón MVVM aplicado

La aplicación sigue una separación estricta entre la View (Fragment), el ViewModel y el Repository. Esto permite que:

- Los Fragments solo se ocupen de mostrar datos y propagar eventos.
- Los ViewModels contengan la lógica de presentación y sobrevivan a los cambios de configuración.
- Los Repositories sean la única fuente de verdad para los datos, ocultando si vienen de red, caché o BD local.

La estructura típica de un módulo (por ejemplo motos nuevas) es:

model/MotoNueva.kt	# Data class
model/MotoNuevaRequest.kt	# DTO de entrada
network/ApiService.kt	# Endpoints Retrofit
repository/MotoNuevaRepository.kt	# Capa de red
viewmodel/MotoNuevaViewModel.kt	# Lógica UI
viewmodel/MotoNuevaViewModelFactory.kt	# Constructor del VM
view/fragment/MotosNuevasListaFragment.kt	# Pantalla lista
view/fragment/MotoNuevaDetalleFragment.kt	# Pantalla detalle
view/fragment/AnadirMotoNuevaFragment.kt	# Form alta/edición
view/fragment/GestionarImagenesMotoNuevaFragment.kt	# Subida media
adapter/MotoNuevaAdapter.kt	# RecyclerView

2. Comunicación con la API

Retrofit se configura en RetrofitClient con la URL base centralizada en Constants.BASE_URL. La autenticación se inyecta automáticamente con un AuthInterceptor de OkHttp que lee el token JWT del SessionManager:

```

object RetrofitClient {

    fun getInstance(context: Context): ApiService {
        val session = SessionManager(context)

        val logging = HttpLoggingInterceptor().apply {
            level = HttpLoggingInterceptor.Level.BODY
        }

        val client = OkHttpClient.Builder()
            .addInterceptor(interceptor = AuthInterceptor(session))
            .addInterceptor(interceptor = logging)
            .build()

        return Retrofit.Builder()
            .baseUrl(baseUrl = Constants.BASE_URL)
            .client(client)
            .addConverterFactory(factory = GsonConverterFactory.create())
            .build()
            .create(ApiService::class.java)
    }
}

```

Tras el login el token se persiste con `SessionManager.saveToken(token)`. El `AuthInterceptor` lo recupera y añade la cabecera `Authorization: Bearer <token>` a cada petición saliente. Si no hay token (usuario invitado o aún no logueado), la petición sale sin cabecera y solo los endpoints `permitAll` responden con éxito.

3. Gestión de imágenes con Glide

Glide se utiliza para cargar todas las imágenes de la app, gestionando caché en memoria y disco. Para evitar parpadeos al cambiar de imagen rápidamente (sobre todo en la vista 360), se usan dos opciones clave:

```

// Precarga todas las imagenes 360
val baseUrl = com.example.motos.utils.Constants.BASE_URL.removeSuffix(suffix = "/")
list.forEach { img ->
    Glide.with(fragment = this).load(string = "$baseUrl/uploads/imagenes/${img.url}").preload()
}

```

4. Reproductor de vídeo con ExoPlayer

Inicialmente se intentó usar YouTube embebido vía WebView, pero los canales oficiales bloquean el embed con error 152/153. La solución definitiva es subir vídeos al propio servidor y reproducirlos con ExoPlayer (androidx.media3):

```
player = androidx.media3.exoplayer.ExoPlayer.Builder(context = requireContext()).build().apply {
    binding.playerVideo.player = this
    setMediaItem(androidx.media3.common.MediaItem.fromUri(uri = url))
    prepare()
    playWhenReady = false
}

binding.playerVideo.setFullscreenButtonClickListener { isFullscreen ->
    toggleFullscreen(isFullscreen)
}
```

El modo pantalla completa custom saca el PlayerView del layout y lo añade al decorView con MATCH_PARENT, fondo negro y bandas de inmersión:

```
private fun toggleFullscreen(isFullscreen: Boolean) {
    val activity = requireActivity()
    val decor = activity.window.decorView as ViewGroup

    if (isFullscreen) {
        originalSystemUi = activity.window.decorView.systemUiVisibility

        contenedorOriginal = binding.playerVideo.parent as? ViewGroup
        indiceOriginal = contenedorOriginal?.indexOfChild(child = binding.playerVideo) ?: -1
        paramsOriginales = binding.playerVideo.layoutParams

        contenedorOriginal?.removeView(view = binding.playerVideo)
        binding.playerVideo.setBackgroundColor(android.graphics.Color.BLACK)
        decor.addView(
            child = binding.playerVideo,
            params = ViewGroup.LayoutParams(
                width = ViewGroup.LayoutParams.MATCH_PARENT,
                height = ViewGroup.LayoutParams.MATCH_PARENT
            )
        )

        activity.window.decorView.systemUiVisibility = (
            View.SYSTEM_UI_FLAG_FULLSCREEN
            or View.SYSTEM_UI_FLAG_HIDE_NAVIGATION
            or View.SYSTEM_UI_FLAG_IMMERSIVE_STICKY
        )
    } else {
        decor.removeView(view = binding.playerVideo)
        binding.playerVideo.setBackgroundColor(android.graphics.Color.TRANSPARENT)
        contenedorOriginal?.addView(child = binding.playerVideo, index = indiceOriginal, paramsOriginales)

        activity.window.decorView.systemUiVisibility = originalSystemUi
    }

    enFullscreen = isFullscreen
}
```

5. Vista 360°

La vista 360 se implementa con un único ImageView y un onTouchListener que cambia el frame mostrado según el desplazamiento horizontal del dedo:

```
private fun configurarTouch360() {
    var inicioX = 0f
    var frameInicio = 0
    binding.ivPrincipal.setOnTouchListener { _, event ->
        if (!modo360 || imagenes360.isEmpty()) return@setOnTouchListener false
        when (event.action) {
            MotionEvent.ACTION_DOWN -> {
                inicioX = event.x
                frameInicio = frame360
                true
            }
            MotionEvent.ACTION_MOVE -> {
                val deltaX = event.x - inicioX
                val sensibilidad = binding.ivPrincipal.width / imagenes360.size.toFloat()
                val delta = (deltaX / sensibilidad).toInt()
                val nuevo = frameInicio - delta
                if (nuevo != frame360) {
                    frame360 = nuevo
                    mostrarFrame360()
                }
                true
            }
        }
        else -> false
    }
}
```

6. Búsqueda tolerante (Levenshtein)

El filtro de motos de segunda mano implementa una búsqueda inteligente que tolera errores tipográficos. Se basa en el algoritmo de distancia de Levenshtein y varias heurísticas adicionales (subcadena, prefijo, palabra a palabra).

La función coincideTolerante combina las distintas estrategias en cascada, ordenadas de más rápida a más costosa:

```

2 Usages
private fun coincideTolerante(texto: String, query: String): Boolean {
    val txt = texto.lowercase()
    if (txt.contains(other = query)) return true
    // Comprueba palabras del texto contra el query
    return txt.split(...delimiters = " ").any { palabra ->
        distanciaLevenshtein(a = palabra, b = query) <= 2 ||
        palabra.startsWith(prefix = query) ||
        query.startsWith(prefix = palabra.take(n = 3))
    } || distanciaLevenshtein(a = txt, b = query) <= 2
}

```

La distancia de Levenshtein se calcula con programación dinámica $O(m \cdot n)$:

```

2 Usages
private fun distanciaLevenshtein(a: String, b: String): Int {
    val m = a.length
    val n = b.length
    if (m == 0) return n
    if (n == 0) return m
    val dp = Array(size = m + 1) { IntArray(size = n + 1) }
    for (i in 0 ≤ .. ≤ m) dp[i][0] = i
    for (j in 0 ≤ .. ≤ n) dp[0][j] = j
    for (i in 1 ≤ .. ≤ m) {
        for (j in 1 ≤ .. ≤ n) {
            val cost = if (a[i - 1] == b[j - 1]) 0 else 1
            dp[i][j] = minOf(
                a = dp[i - 1][j] + 1,
                b = dp[i][j - 1] + 1,
                c = dp[i - 1][j - 1] + cost
            )
        }
    }
    return dp[m][n]
}

```

BORRAR
 INSERTAR
 SUSTITUIR

Con un umbral máximo de 2 errores, el algoritmo acepta typos como Kasawaki en vez de Kawasaki o Hnda en vez de Honda.

6. Decisiones técnicas relevantes

1. Por qué JWT con BCrypt

Se ha optado por JSON Web Tokens en lugar de HTTP Basic por varias razones:

- Seguridad: con HTTP Basic la contraseña viaja en cada petición codificada en Base64 (no cifrada). Con JWT solo viaja una vez (en login) y a partir de ahí solo viaja un token con caducidad.
- Stateless: el backend no mantiene sesión por usuario. Cada petición se autentica por sí misma con el token, lo que permite escalar horizontalmente sin replicar sesiones entre instancias.
- Estándar moderno: JWT es el estándar de facto en APIs REST y aplicaciones móviles actuales.
- Información embebida: el token contiene el username y el rol como claims, evitando consultas a BD en cada petición.

Las contraseñas se almacenan cifradas con BCrypt, un algoritmo de hash adaptativo con sal automática. Si la base de datos fuera comprometida, las contraseñas seguirían siendo inviables de descifrar por fuerza bruta.

Como mejora futura, se podría añadir refresh tokens para renovar la sesión sin que el usuario vuelva a introducir sus credenciales, y revocar tokens individuales en caso de robo.

2. ¿Por qué Docker Compose?

Permite levantar todo el entorno (backend + BD) con un único comando, garantizando que el desarrollo, la defensa y cualquier despliegue futuro arranquen exactamente igual. También facilita probar en cualquier máquina con Docker instalado.

3. ¿Por qué ngrok?

La conexión doméstica del autor utiliza CG-NAT, lo que impide abrir puertos en el router. ngrok crea un túnel HTTPS público sin necesidad de tocar la configuración de red, ofreciendo además un dominio fijo en el plan gratuito. Es la solución más simple para una demo presencial.

4. ¿Por qué ExoPlayer en lugar de YouTube?

YouTube aplica restricciones de embed muy variables según el canal y la región. Vídeos de Yamaha, Honda y otros canales oficiales bloquean el embed devolviendo error 152 o 153. Servir los vídeos desde el propio servidor con ExoPlayer da control total y evita estos problemas.

1. ¿Por qué OpenMultipleDocuments en lugar de GetMultipleContents?

El selector nativo de Android 13+ (Photo Picker) limita la selección múltiple a 5 elementos. OpenMultipleDocuments abre el explorador del sistema (DocumentsUI), que no tiene ese límite, permitiendo subir tantas imágenes como se quiera de una vez.

5. ¿Por qué desactivar Auto Backup?

Android restaura automáticamente las SharedPreferences al reinstalar una app. Eso provocaba que la sesión se mantuviera tras desinstalar. Para tener un comportamiento determinista en demos, se ha desactivado con `allowBackup="false"` y `fullBackupContent="false"`.

Guia del programador

7. Configuración del entorno de desarrollo

1. Software necesario

- Android Studio Iguana o superior para la app móvil / backend.
- Docker Desktop con Compose v2.
- MySQL Workbench o DBeaver (opcional, para inspeccionar la BD).
- Postman (para probar endpoints manualmente).
- ngrok (cuenta gratuita registrada y authToken configurado).

1. Clonar el proyecto

```
git clone <https://github.com/iesgrancapitan-proyectos/202526DAM-mayo-Taller-motos.git>
cd proyecto
```

2. Levantar el backend en modo desarrollo

Docker:

```
docker compose up --build
```

3. Abrir la app Android

- Abrir Android Studio.
- File → Open → seleccionar la carpeta motos/.
- Esperar a que Gradle sincronice.
- Comprobar que `Constants.BASE_URL` apunta a la URL correcta (ngrok o localhost en emulador 10.0.2.2).
- Pulsar Run.

8. Cómo añadir un nuevo módulo completo

Esta sección describe paso a paso el proceso para añadir un nuevo recurso al sistema (por ejemplo, un módulo de Accesorios). Se incluyen tanto los cambios en el backend como en la app Android.

Crear el archivo `model/Accesorio.kt`:

```
@Entity
@Table(name = "accesorio")
data class Accesorio(
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    val id: Long = 0,
    val nombre: String,
    val descripcion: String = "",
    val precio: Double,
    val stock: Int = 0
)
```

1. Backend: repositorio

```
interface AccesorioRepository : JpaRepository<Accesorio, Long> {
    fun findByNombreContainingIgnoreCase(nombre: String): List<Accesorio>
}
```

2. Backend: DTO

```
data class AccesorioRequest(
    val nombre: String,
    val descripcion: String = "",
    val precio: Double,
    val stock: Int = 0
)
```

3. Backend: servicio

```
@Service
class AccesorioService(private val repo: AccesorioRepository) {
    fun listar(): List<Accesorio> = repo.findAll()
    fun buscar(q: String) = repo.findByNombreContainingIgnoreCase(q)
    fun crear(req: AccesorioRequest) = repo.save(
        Accesorio(nombre = req.nombre, descripcion = req.descripcion,
            precio = req.precio, stock = req.stock)
    )
    fun actualizar(id: Long, req: AccesorioRequest): Accesorio {
        val orig = repo.findById(id).orElseThrow()
        return repo.save(orig.copy(
            nombre = req.nombre, descripcion = req.descripcion,
            precio = req.precio, stock = req.stock))
    }
    fun eliminar(id: Long) = repo.deleteById(id)
}
```

4. Backend: controlador

```
@RestController
@RequestMapping("/accesorios")
class AccesorioController(private val service: AccesorioService) {
    @GetMapping fun listar() = service.listar()
    @GetMapping("/buscar") fun buscar(@RequestParam q: String) = service.buscar(q)
    @PostMapping fun crear(@RequestBody r: AccesorioRequest) = service.crear(r)
    @PutMapping("/{id}")
    fun actualizar(@PathVariable id: Long, @RequestBody r: AccesorioRequest) =
        service.actualizar(id, r)
    @DeleteMapping("/{id}") fun eliminar(@PathVariable id: Long) = service.eliminar(id)
```

1. Backend: configurar seguridad

En SecurityConfig.kt, añadir las rutas según el rol que pueda acceder:

```
.requestMatchers(HttpMethod.GET, "/accesorios/**").permitAll()
.requestMatchers("/accesorios/**")
    .hasAnyRole("ADMIN", "VENDEDOR")
```

5. Android: data class y endpoints

```
data class Accesorio(
    val id: Long = 0L,
    val nombre: String,
    val descripcion: String = "",
    val precio: Double,
    val stock: Int = 0
)

@GET("accesorios")
suspend fun getAccesorios(): Response<List<Accesorio>>
@POST("accesorios")
suspend fun crearAccesorio(@Body req: AccesorioRequest): Response<Accesorio>
@PUT("accesorios/{id}")
suspend fun actualizarAccesorio(@Path("id") id: Long,
    @Body req: AccesorioRequest): Response<Accesorio>
@DELETE("accesorios/{id}")
suspend fun eliminarAccesorio(@Path("id") id: Long): Response<Unit>
```

6. Android: repositorio, ViewModel y Fragments

Replicar el mismo patrón que en MotoNuevaRepository, MotoNuevaViewModel y MotoNuevaListaFragment cambiando los nombres y los endpoints. La estructura es siempre la misma.

9. Cómo añadir una nueva pantalla en Android

Pasos para añadir un Fragment con su layout, ViewModel y conexión a la API:

- Crear el archivo XML del layout en res/layout/. Por convención, el nombre empieza por fragment_<nombre>.xml.
- Crear la clase Fragment en view/fragment/. Heredar de Fragment y usar View Binding para acceder a las vistas.
- Definir el ViewModel correspondiente en viewmodel/ con LiveData para los datos.
- Crear el ViewModelFactory si el ViewModel necesita parámetros (la mayoría de los casos).
- En el Repository, añadir las funciones suspend que llamen al ApiService.
- En el Fragment, en onCreateView, observar los LiveData y configurar listeners.
- Añadir la navegación desde el Fragment de origen, usando FragmentTransaction.replace().
-

Plantilla mínima del Fragment:

```
class MiNuevoFragment : Fragment() {
    private var _binding: FragmentMiNuevoBinding? = null
    private val binding get() = _binding!!
    private val viewModel: MiNuevoViewModel by viewModels {
        MiNuevoViewModelFactory(
            MiNuevoRepository(RetrofitClient.getInstance(requireContext()))
        )
    }
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentMiNuevoBinding.inflate(inflater, container, false)
        return binding.root
    }
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        viewModel.datos.observe(viewLifecycleOwner) { lista ->
            // actualizar UI
        }
        viewModel.cargar()
    }
    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Modificar el modelo de datos

1. Cambios sin pérdida de datos

Cuando ddl-auto está en update, Hibernate aplica automáticamente cambios no destructivos:

- Añadir nuevas columnas (con valor por defecto).
- Añadir nuevas tablas o índices.
- Aumentar el tamaño de columnas VARCHAR.

2. Cambios destructivos

Hibernate NO realiza estos cambios automáticamente. Hay que ejecutar manualmente las sentencias SQL:

- Borrar columnas o tablas.
- Renombrar columnas o tablas.
- Cambiar tipo de columna (por ejemplo, de VARCHAR a INT).

Para hacer estos cambios:

- Acceder al contenedor: `docker exec -it concesionario-db mysql -uroot -proot concesionario`
- Ejecutar las sentencias DDL necesarias.
- Reiniciar el backend para que detecte el nuevo esquema.

1. Borrar todos los datos y empezar de cero

La forma más rápida es:

```
docker compose down -v # -v borra los volúmenes
docker compose up --build
```

10. Buenas prácticas y debugging

1. Logs útiles

Backend: Spring Boot muestra todas las queries de Hibernate si se activa esta línea en application.properties:

```
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Android: para ver las peticiones HTTP de Retrofit, OkHttp tiene un Interceptor de logging. Añadirlo solo en debug:

```
if (BuildConfig.DEBUG) {
    val logging = HttpLoggingInterceptor().apply {
        level = HttpLoggingInterceptor.Level.BODY
    }
    builder.addInterceptor(logging)
}
```

2. Errores frecuentes

1. Cleartext HTTP not permitted

Si la BASE_URL es http:// (no https://), hay que activar tráfico no cifrado en AndroidManifest.xml:

```
<application
    android:usesCleartextTraffic="true"
    ... />
```

2. Error 401/403 al acceder a un endpoint

- Comprobar que el usuario tiene el rol correcto.
- Verificar que SessionManager guarda y recupera bien las credenciales.
- Revisar que SecurityConfig permita el rol de la petición.

3. Error 500 al subir archivos

- Confirmar que la carpeta /app/uploads existe en el contenedor.
- Subir el límite de Spring si el archivo supera 200MB.
- Revisar logs con docker compose logs -f.

4. OutOfMemoryError al cargar imágenes

- Glide ya gestiona el bitmap pool, pero si las imágenes son muy grandes (>10 MB) usar .override(width, height) para redimensionarlas.
- Considerar comprimir las imágenes antes de subirlas al servidor.

11. Despliegue

1. Generar APK firmado de release

- En Android Studio: Build → Generate Signed App Bundle / APK...
- Seleccionar APK.
- Crear o seleccionar el keystore (archivo .jks con contraseña).
- Elegir variant=release y pulsar Finish.
- El APK se genera en app/release/app-release.apk.